

Table of contents

Model Checking	1
Introduction au Model Checking	1
Le problème du Model Checking	1
Mots infinis et langages -réguliers	2
-mots	2
-expressions régulières	2
Langages -réguliers	3
Problème du vide pour les langages -réguliers	3
Automates de Büchi	4
Définition formelle	4
Sémantique d'acceptation	4
Exemple	4
Logique Temporelle Linéaire (LTL)	5
Syntaxe	6
Sémantique	6
Opérateurs dérivés	6
LTL et automates de Büchi	7
Application au Model Checking	10
Méthode générale	10
Interprétation des résultats	11
Vérification du vide	11
Exemple simplifié : un ascenseur	11
Résumé et perspectives	17

Model Checking

Introduction au Model Checking

Le model checking est une technique formelle de vérification de systèmes, proposée en 1981 par Clarke, Emerson et Sifakis. Elle permet de vérifier automatiquement si un système (modélisé par un automate d'états fini) satisfait une propriété spécifiée en logique temporelle.

Le problème du Model Checking

Étant donné un automate fini A (avec éventuellement des étiquettes) et une formule de logique temporelle F , le problème consiste à :

- Déterminer si $A \models F$, c'est-à-dire si tous les traces de A sont inclus dans les traces de F : $Traces(A) \subseteq Traces(F)$.

- Si ce n'est pas le cas, fournir un contre-exemple : une exécution de A qui ne satisfait pas F .

La difficulté du problème (décidabilité, complexité) dépend de l'expressivité de A et F . Dans ce cours, nous nous concentrons sur :

- A : un automate fini avec uniquement des actions (étiquettes simples) sur les transitions, et rien sur les états.
 - F : une formule de logique temporelle linéaire (LTL).
-

Mots infinis et langages -réguliers

-mots

Les systèmes réactifs ne s'arrêtent pas, donc leurs exécutions sont infinies. De même, les propriétés à vérifier portent sur des comportements infinis. Une trace infini (ou mot infini) d'actions est appelé un **-mot**.

Exemple : Le mot infini constitué de la répétition infinie de "ab" s'écrit $(ab)^\omega$.

À comparer avec $(ab)^*$ qui représente les mots finis avec un nombre arbitraire (fini) de répétitions de "ab". Le symbole ω indique une répétition infinie.

-expressions régulières

En étendant les expressions régulières avec l'opérateur ω , on définit les **-expressions régulières**.

Exemples : Sur l'alphabet $\{a, b\}$:

- $(a + b)^*.a^\omega$ représente les mots infinis avec un nombre fini de b .
- $(a^*b)^\omega$ représente les mots infinis avec un nombre infini de b .

Attention : $a^\omega b^\omega$ n'a pas de sens (on ne peut pas finir de lire les a infinis pour passer aux b) et est donc interdit.

Langages ω -réguliers

De même que les langages réguliers peuvent être représentés par des expressions régulières ou des automates finis, les **langages ω -réguliers** sont exactement ceux qui peuvent être exprimés par :

- des ω -expressions régulières (avec les opérateurs $+$, $\cdot$, $*$, ω),
- des automates de Büchi.

Définition : Un langage L est ω -régulier s'il peut s'écrire sous l'une des formes suivantes (et leurs combinaisons) :

1. $L = A^\omega$ où A est un langage régulier et $\epsilon \notin A$,
2. $L = A \cdot B$ où A est régulier et B est ω -régulier,
3. $L = A \cup B$ où A et B sont ω -réguliers.

Propriétés de clôture : Si on a des automates (finis ou de Büchi) pour A et B , on peut construire un automate de Büchi pour A^ω , $A \cdot B$, $A + B$, $A \cap B$. Cependant, la complémentation $\overline{A}$ est plus complexe (taille exponentielle).

Remarque importante : Contrairement aux automates finis, un automate de Büchi non déterministe ne peut pas toujours être déterminisé. Par exemple, $(a + b)^* a^\omega$ ne peut pas être reconnu par un automate de Büchi déterministe.

Problème du vide pour les langages ω -réguliers

Le problème de décider si un langage ω -régulier L est vide est **décidable**. Étant donné un automate de Büchi A tel que $L(A) = L$, on a :

$L \neq \emptyset$ si et seulement si A contient un ω -mot. Comme le nombre d'états finaux est fini, tout ω -mot accepté est ultimement périodique.

Ainsi, $L \neq \emptyset$ ssi A contient une composante fortement connexe (CFC) atteignable depuis l'état initial et contenant un état final.

Automates de Büchi

Définition formelle

Un automate de Büchi est un tuple $\mathcal{A} = (S, S_0, L, T, F)$ où :

- S : ensemble fini d'états,
- $S_0 \subseteq S$: états initiaux,
- L : alphabet (labels),
- $T \subseteq S \times L \times S$: relation de transition,
- $F \subseteq S$: états finaux (ou d'acceptation).

Sémantique d'acceptation

Un ω -mot $w = a_0 a_1 a_2 \dots$ est accepté par $\mathcal{A}$ s'il existe une exécution infinie $s_0 s_1 s_2 \dots$ telle que :

- $s_0 \in S_0$,
- pour tout $i \geq 0$, $(s_i, a_i, s_{i+1}) \in T$,
- et **un état final** $f \in F$ **apparaît infiniment souvent** dans l'exécution.

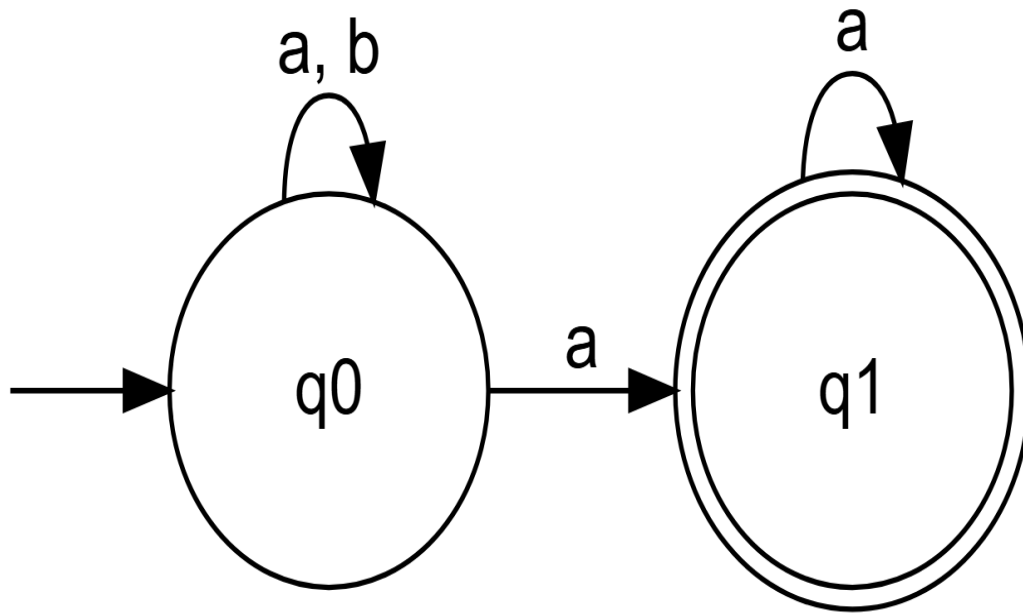
Le langage reconnu par $\mathcal{A}$, noté $L(\mathcal{A})$, est l'ensemble des ω -mots acceptés.

Remarque : Cette condition d'acceptation (visiter infiniment souvent un état final) est caractéristique des automates de Büchi.

Voici un exemple d'automate de Büchi illustrant le langage ω -régulier $(a+b)^* a^\omega$ (les mots infinis contenant un nombre fini de b), intégré directement dans votre cours.

Exemple

L'automate de Büchi ci-dessous reconnaît le langage ω -régulier $L = (a+b)^* a^\omega$, c'est-à-dire l'ensemble des mots infinis sur $\{a, b\}$ qui, après un certain préfixe fini, ne contiennent plus que le symbole a .



Fonctionnement de l'automate :

- Depuis l'état initial q_0 , l'automate peut lire n'importe quelle séquence finie (éventuellement vide) de a et de b en restant dans q_0 (transition sur a, b).
- À tout moment, en lisant un a , il peut choisir de passer dans l'état final q_1 .
- Une fois dans q_1 , il ne peut lire que des a (boucle sur a) pour y rester.
- Pour qu'un mot infini soit **accepté**, il doit exister une exécution qui visite q_1 infiniment souvent. La seule façon de faire cela est de passer dans q_1 à un moment donné et de n'y lire ensuite que des a indéfiniment. Par conséquent, tout mot accepté doit finir par une infinité de a , ce qui correspond exactement à la définition du langage $(a + b)^* a^\omega$.

Cet exemple illustre également la **non-déterministe** de certains automates de Büchi : à la lecture d'un a dans q_0 , l'automate a le choix entre rester dans q_0 ou aller dans q_1 . Ce non-déterminisme est essentiel pour reconnaître ce langage avec un automate de Büchi.

Logique Temporelle Linéaire (LTL)

Introduite par Pnueli en 1977, la LTL permet d'exprimer des propriétés sur les comportements infinis des systèmes.

Syntaxe

La syntaxe de LTL est définie par la grammaire suivante :

$$\phi ::= a \mid \neg\phi \mid \phi_1 \vee \phi_2 \mid \bigcirc\phi \mid \phi_1 \mathcal{U} \phi_2$$

où a est une variable propositionnelle (une action ou une propriété atomique).

Sémantique

Une formule LTL ϕ interprète un -mot $\sigma = (\sigma_i)_{i \geq 0}$ (où chaque σ_i est un ensemble de propositions vraies à l'instant i). La relation de satisfaction $\sigma \models \phi$ est définie inductivement :

- $\sigma \models a$ ssi $a \in \sigma_0$ (la proposition a est vraie au premier instant).
- $\sigma \models \neg\phi$ ssi $\sigma \not\models \phi$.
- $\sigma \models \phi_1 \vee \phi_2$ ssi $\sigma \models \phi_1$ ou $\sigma \models \phi_2$.
- $\sigma \models \bigcirc\phi$ ssi $\sigma' = (\sigma_i)_{i \geq 1} \models \phi$ (la formule ϕ est vraie à l'instant suivant).
- $\sigma \models \phi_1 \mathcal{U} \phi_2$ ssi il existe $j \geq 0$ tel que :
 - $\sigma^{(j)} = (\sigma_i)_{i \geq j} \models \phi_2$,
 - et pour tout k avec $0 \leq k < j$, on a $\sigma^{(k)} = (\sigma_i)_{i \geq k} \models \phi_1$.

Intuitivement, $\phi_1 \mathcal{U} \phi_2$ signifie que ϕ_1 reste vraie jusqu'à ce que ϕ_2 devienne vraie (et ϕ_2 finit par devenir vraie).

Opérateurs dérivés

À partir des opérateurs de base, on définit :

- **Éventuellement** : $\Diamond\phi \equiv \text{True} \mathcal{U} \phi$. La formule ϕ devient vraie après un temps fini.
- **Toujours** : $\Box\phi \equiv \neg\Diamond\neg\phi$. La formule ϕ est vraie à tout instant.

Quelques équivalences utiles :

- $\Box(a \vee b) \neq \Box a \vee \Box b$
- $\Diamond(a \vee b) \equiv \Diamond a \vee \Diamond b$
- $\Box(a \wedge b) \equiv \Box a \wedge \Box b$
- $\Diamond(a \wedge b) \neq \Diamond a \wedge \Diamond b$ (car a et b peuvent se produire à des instants différents).

LTL et automates de Büchi

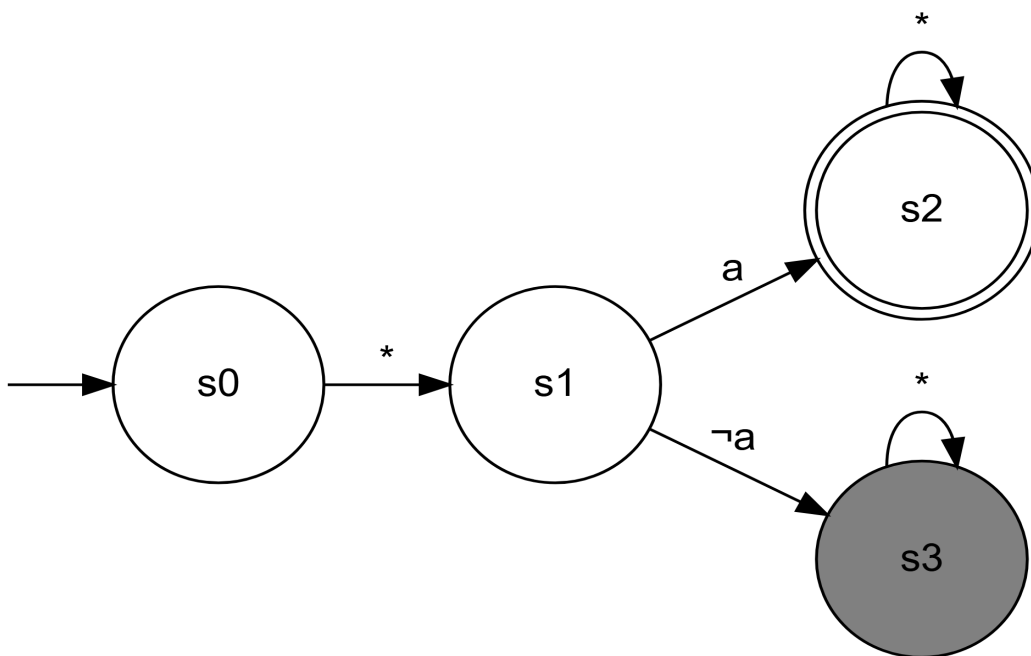
Un résultat fondamental : toute formule LTL peut être traduite en un automate de Büchi qui accepte exactement les ω -mots satisfaisant la formule. Il existe des algorithmes pour cette construction.

Automates de Büchi pour les opérateurs LTL de base

Pour mieux comprendre la sémantique des opérateurs LTL, voici des automates de Büchi qui reconnaissent exactement les ω -mots satisfaisant chaque formule.

1. Opérateur Next : $\bigcirc a$

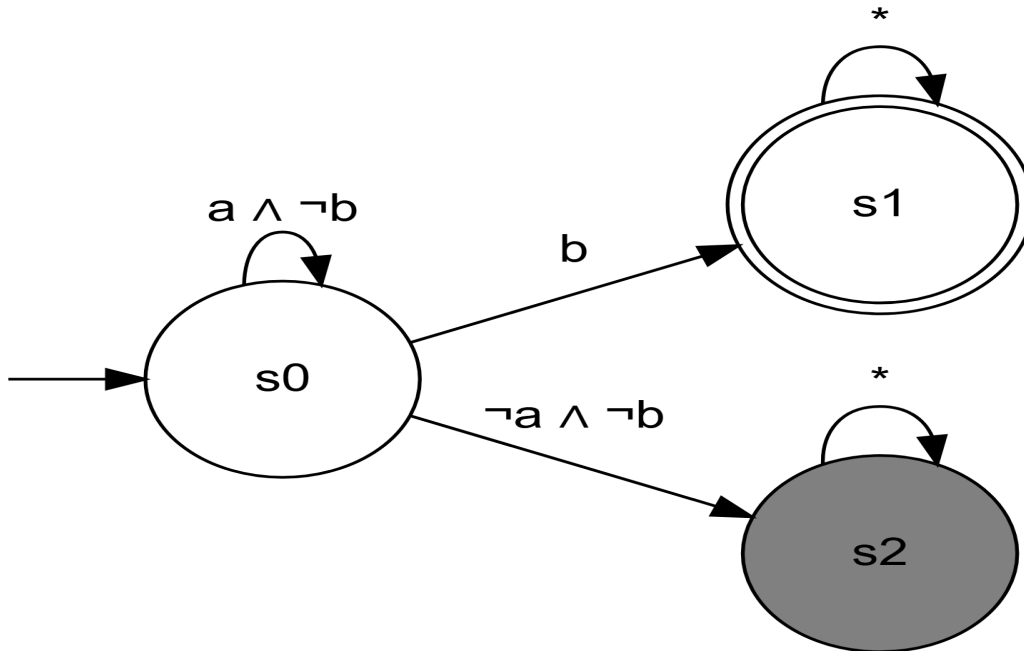
La formule $\bigcirc a$ exige que a soit vrai à l'instant suivant l'instant courant.



Fonctionnement : L'automate passe immédiatement de l'état initial s_0 à s_1 (représentant l'instant suivant). Si a est vrai dans cet état suivant (s_1), il va dans l'état final s_2 et le mot est accepté. Si a est faux, il entre dans l'état puits s_3 et n'est pas accepté.

2. Opérateur Until : $a \mathcal{U} b$

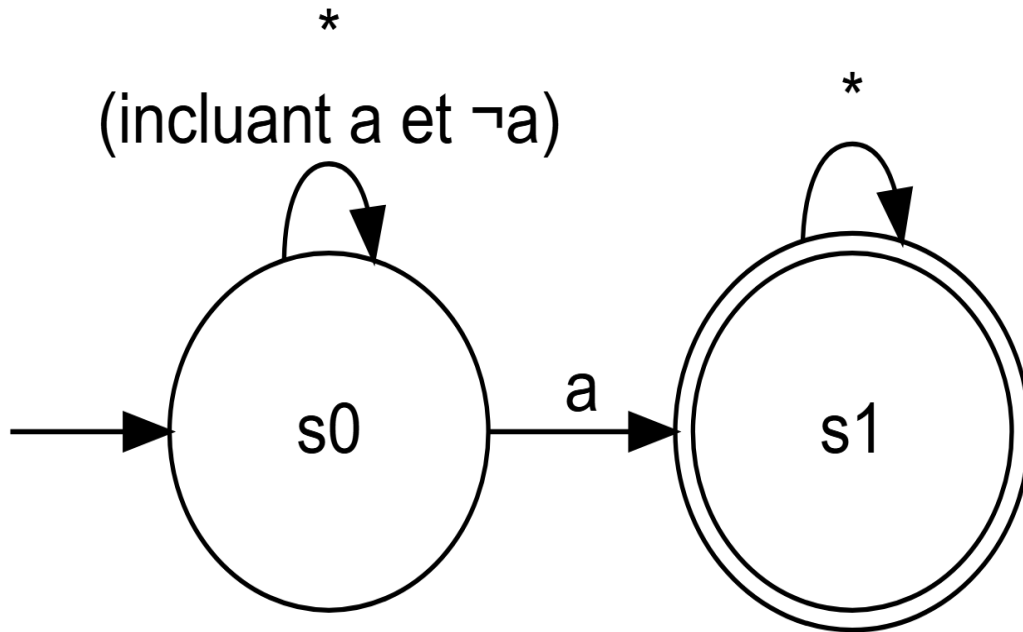
La formule $a \mathcal{U} b$ exige que a reste vrai jusqu'à ce que b devienne vrai (et b doit finir par devenir vrai).



Fonctionnement : Tant que $a \wedge \neg b$ est vrai, l'automate reste dans l'état initial s_0 . Dès que b devient vrai, il passe à l'état final s_1 (et y reste) et le mot est accepté. Si $\neg a \wedge \neg b$ devient vrai, il entre dans l'état puits s_2 et le mot n'est pas accepté.

3. Opérateur Eventually : $\Diamond a$

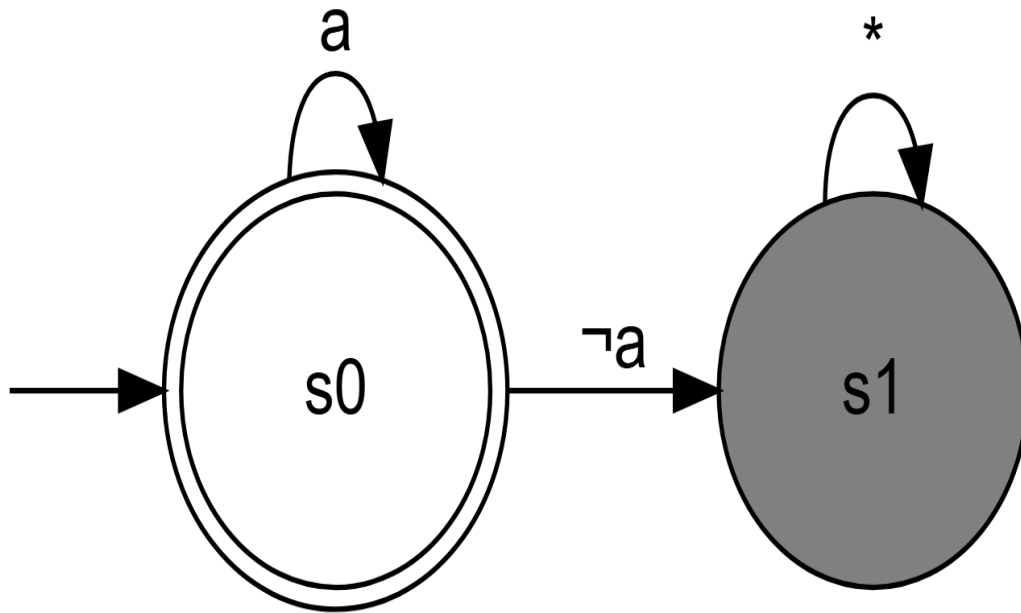
La formule $\Diamond a$ exige que a devienne vrai à un moment futur.



Fonctionnement : L'automate reste dans l'état initial s_0 . À un moment donné, lorsque a devient vrai (n'excluant pas que d'autres a ont pu se produire), il passe dans l'état final s_1 et y reste, acceptant le mot. Si a n'est jamais vrai, il reste indéfiniment dans s_0 et le mot n'est pas accepté (car s_0 n'est pas final).

4. Opérateur Always : $\Box a$

La formule $\Box a$ exige que a soit vrai à tout instant.



Fonctionnement : L'automate commence dans l'état final s_0 . Tant que a est vrai, il reste dans s_0 . Dès que a devient faux, il entre dans l'état puits s_1 et le mot n'est pas accepté. Pour être accepté, a doit donc être vrai à chaque instant.

Ces automates illustrent comment la sémantique de chaque opérateur se traduit en conditions d'acceptation sur les exécutions infinies. Ils constituent la base de la construction systématique d'automates pour des formules LTL plus complexes.

Application au Model Checking

Méthode générale

Pour vérifier qu'un système A (modélisé par un automate de Büchi) satisfait une propriété F (exprimée en LTL), on suit les étapes suivantes :

1. Modéliser le système par un automate de Büchi A .
2. Exprimer la propriété désirée sous forme d'une formule LTL F .
3. Construire un automate de Büchi pour la négation de F , noté $\neg F$.
4. Calculer le produit synchrone des automates A et $\neg F$, qui accepte le langage $L(A) \cap L(\neg F)$.
5. Vérifier si $L(A) \cap L(\neg F)$ est vide.

Interprétation des résultats

- Si $L(A) \cap L(\neg F) = \emptyset$, alors $L(A) \subseteq L(F)$, ce qui signifie que $A \models F$ (le système satisfait la propriété).
- Sinon, tout -mot dans $L(A) \cap L(\neg F)$ est un **contre-exemple** : une exécution du système A qui viole la propriété F .

Vérification du vide

Pour vérifier si $L(A) \cap L(\neg F)$ est vide, on utilise l'algorithme basé sur les composantes fortement connexes (CFC) :

- Construire le graphe de l'automate produit.
- Chercher une CFC qui soit :
 1. Atteignable depuis un état initial,
 2. Contient au moins un état final.
- Si une telle CFC existe, alors le langage est non vide (il existe un contre-exemple). Sinon, il est vide.

Exemple simplifié : un ascenseur

Considérons un système d'ascenseur modélisé par un automate A . Une propriété de sûreté pourrait être : “si l'ascenseur démarre, alors immédiatement après, il restera en mouvement jusqu'à ce qu'il s'arrête”. En LTL, cela s'exprime :

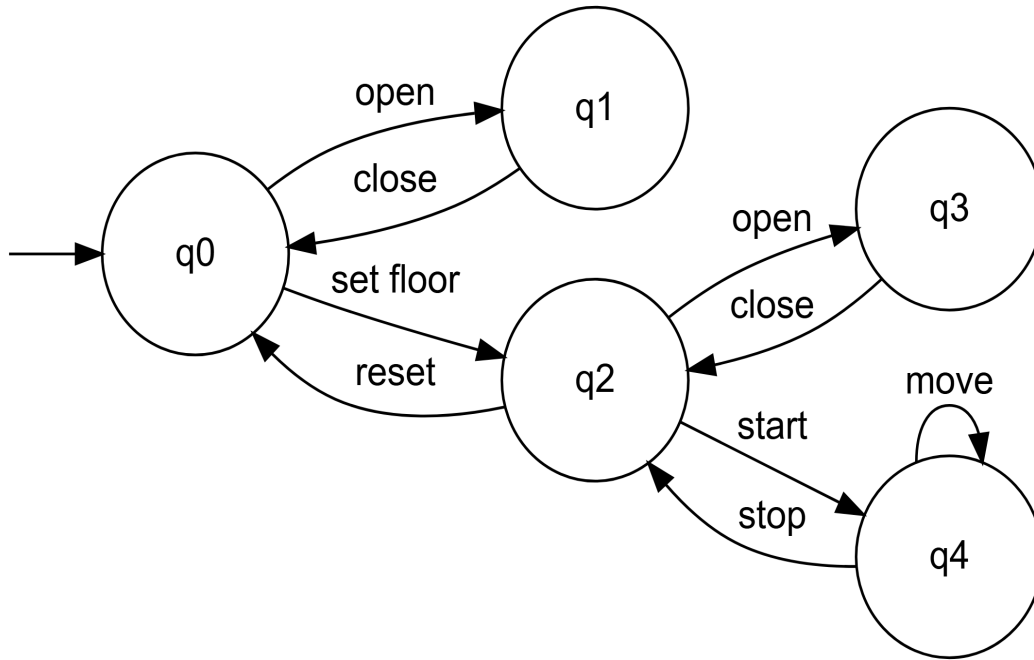
$$F = \Box (\text{start} \Rightarrow \bigcirc (\text{move } \mathcal{U} \text{ stop}))$$

La négation est :

$$\neg F = \Diamond (\text{start} \wedge \bigcirc (\neg(\text{move } \mathcal{U} \text{ stop})))$$

Étape 1 : Modélisation du système

Voici l'automate A représentant le comportement simplifié d'un ascenseur :



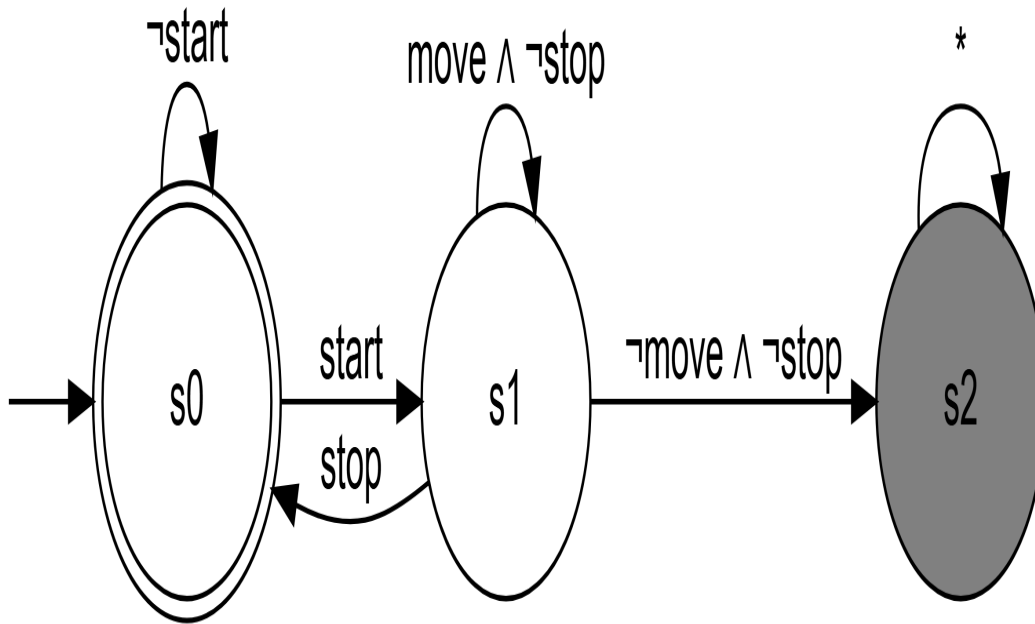
Description des états :

- q_0 : État de repos (portes fermées, aucun étage sélectionné)
- q_1 : Portes ouvertes
- q_2 : Étage sélectionné (attente)
- q_3 : Portes ouvertes après sélection d'étage
- q_4 : Ascenseur en mouvement

Étape 2 : Construction des automates de Büchi pour F et $\neg F$

1. Automate de Büchi pour $F = \Box(\text{start} \Rightarrow \bigcirc(\text{move} \mathcal{U} \text{stop}))$

Cet automate accepte les exécutions où chaque occurrence de **start** est suivie d'un état où **move** reste vrai jusqu'à ce que **stop** devienne vrai.

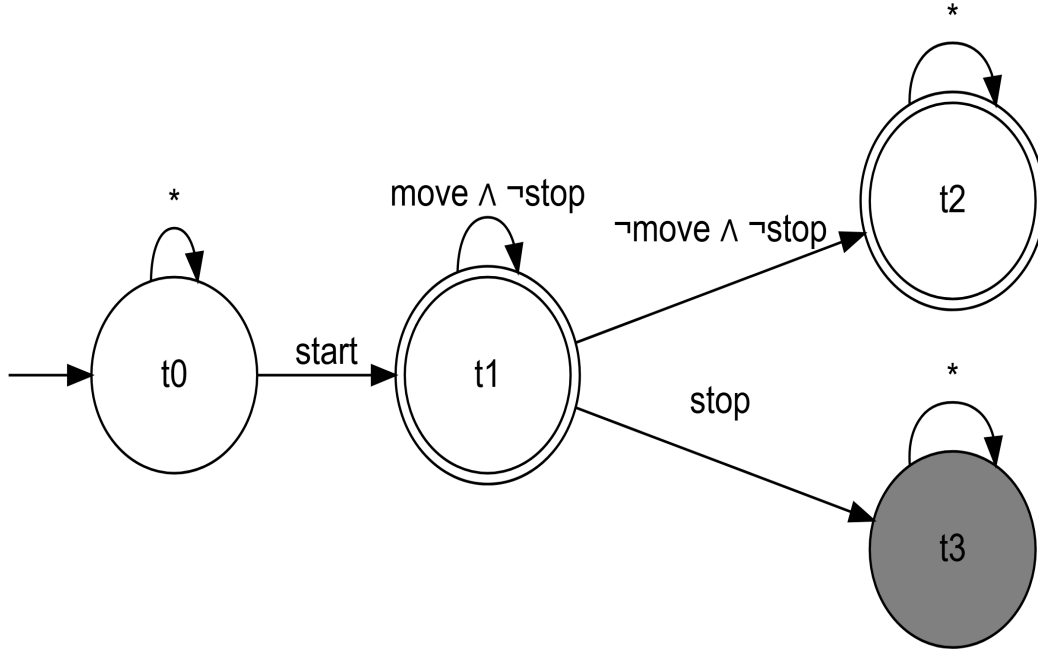


Fonctionnement :

- s_0 (état final) : Aucune obligation en cours
- s_1 : Un **start** vient de se produire, on doit maintenant vérifier $\text{move} \mathcal{U} \text{stop}$
- **Condition d'acceptation** : Visiter s_0 infiniment souvent
- Une exécution n'est acceptée que si chaque obligation $\text{move} \mathcal{U} \text{stop}$ est finalement satisfaite (par un **stop**)

2. Automate de Büchi pour $\neg F = \Diamond(\text{start} \wedge \bigcirc(\neg(\text{move} \mathcal{U} \text{stop})))$

Cet automate accepte les exécutions où à un certain moment, un **start** se produit et immédiatement après, $\text{move} \mathcal{U} \text{stop}$ est faux.



Fonctionnement :

- t_0 : On n'a pas encore vu le **start** problématique
- t_1 : On vient de voir **start**, on passe à l'instant suivant
- t_2 (état final) : On vérifie $\neg(\text{move} \mathcal{U} \text{stop})$; ici tant qu'on a $\text{move} \quad \neg\text{stop}$, on reste dans t_2
- t_3 (état final) : On a $\neg\text{move} \quad \neg\text{stop}$, ce qui viole immédiatement $\text{move} \mathcal{U} \text{stop}$
- **Condition d'acceptation** : Visiter t_2 ou t_3 infiniment souvent
- Une exécution est acceptée si elle reste indéfiniment dans t_2 (avec $\text{move} \quad \neg\text{stop}$) ou dans t_3

Étape 3 : Produit synchrone $A \times \neg F$

Le produit combine les états de A et de $\neg F$. Analysons les exécutions possibles :

Exécution problématique :

1. $q_0 \xrightarrow{\text{set floor}} q_2$
2. $q_2 \xrightarrow{\text{start}} q_4$
3. $q_4 \xrightarrow{\text{move}} q_4 \xrightarrow{\text{move}} q_4 \xrightarrow{\text{move}} \dots$

Cette exécution correspond au mot : **set floor, start, move, move, move, ...**

Dans le produit :

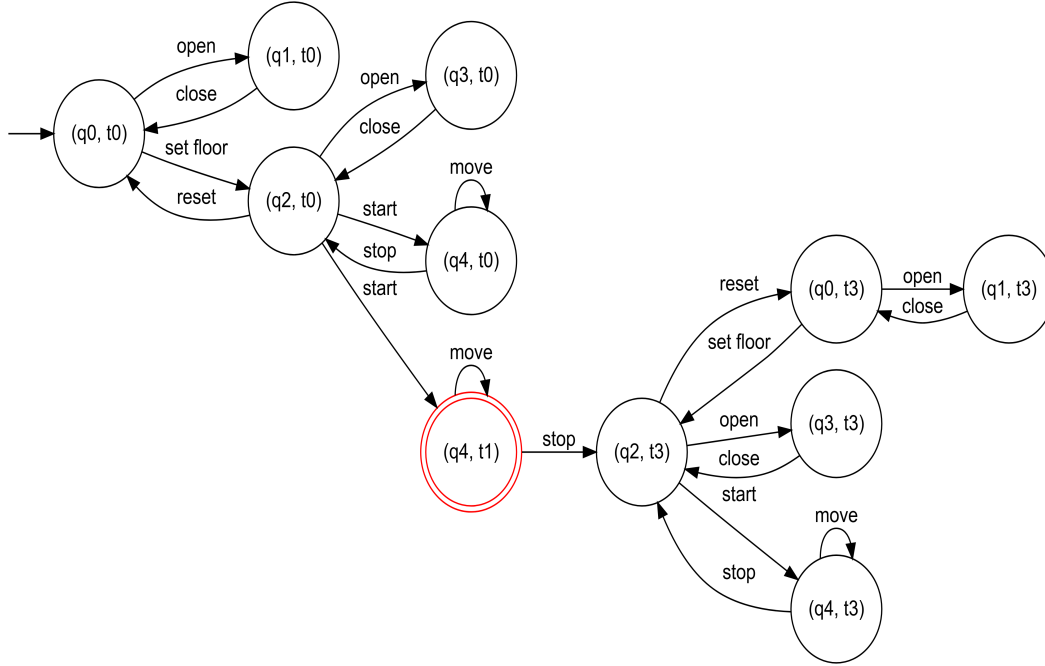
- Après **start**, on passe à t_2 dans $\neg F$
- Dans t_2 , tant qu'on lit **move** $\neg$ **stop**, on reste dans t_2
- Comme l'ascenseur peut rester indéfiniment en q_4 avec **move**, on a une CFC (q_4, t_2) avec la boucle **move**

Cette CFC (q_4, t_2) est :

1. Atteignable depuis l'état initial
2. Contient un état final (t_2 est final)

Étape 3 : Produit synchrone $A \times \neg F$

Le produit synchrone combine les états de A et de $\neg F$. Un état du produit est un couple (q_i, t_j) où q_i est un état du système et t_j un état de l'automate de la propriété violée.



Analyse du produit :

1. **États finals** : Seul (q_4, t_1) est final (représenté par un double cercle rouge), car t_1 est un état final dans $\neg F$.
2. **Composante fortement connexe (CFC) problématique** :
 - L'état (q_4, t_1) forme une CFC avec la transition $(q_4, t_1) \xrightarrow{\text{move}} (q_4, t_1)$
 - Cette CFC est atteignable depuis l'état initial via : $(q_0, t_0) \xrightarrow{\text{set floor}} (q_2, t_0) \xrightarrow{\text{start}} (q_4, t_1)$

- Cette CFC contient un état final (t_1 est final)

Étape 4 : Vérification du vide

Puisqu'il existe une CFC (q_4, t_1) qui :

1. Est atteignable depuis un état initial
2. Contient au moins un état final

Alors $L(A) \cap L(\neg F) \neq \emptyset$.

Étape 5 : Contre-exemple

Le langage n'étant pas vide, on peut extraire un contre-exemple. L'exécution suivante est acceptée par le produit :

1. $(q_0, t_0) \xrightarrow{\text{set floor}} (q_2, t_0)$
2. $(q_2, t_0) \xrightarrow{\text{start}} (q_4, t_1)$
3. $(q_4, t_1) \xrightarrow{\text{move}} (q_4, t_1) \xrightarrow{\text{move}} (q_4, t_1) \xrightarrow{\text{move}} \dots$

Cette exécution correspond au mot infini : `set floor, start, move, move, move, ...`

Interprétation :

- Le système A **ne satisfait pas** la propriété F
- La raison : l'ascenseur peut démarrer et continuer à se déplacer indéfiniment sans jamais exécuter l'action **stop**
- Pour corriger cela, il faudrait modifier le système pour garantir qu'après un **start**, l'ascenseur finisse toujours par exécuter **stop** (par exemple en ajoutant une temporisation ou une surveillance)

Cet exemple illustre comment le model checking LTL permet de détecter des défauts de conception dans les systèmes.

Résumé et perspectives

Le model checking est une méthode automatique et complète (sous les hypothèses d'un système fini) pour vérifier des propriétés temporelles. Les points clés sont :

- La modélisation du système et des propriétés avec des automates de Büchi et la logique LTL.
- La réduction du problème de vérification à un problème de non-vide d'intersection de langages ω -réguliers.
- L'utilisation d'algorithmes sur les graphes (CFC) pour décider le vide.

Cependant, la complexité peut être élevée (notamment à cause de la construction de l'automate pour la négation de la formule LTL). Des techniques d'optimisation et des outils industriels (comme SPIN, NuSMV) sont développés pour appliquer le model checking à des systèmes de taille réelle.